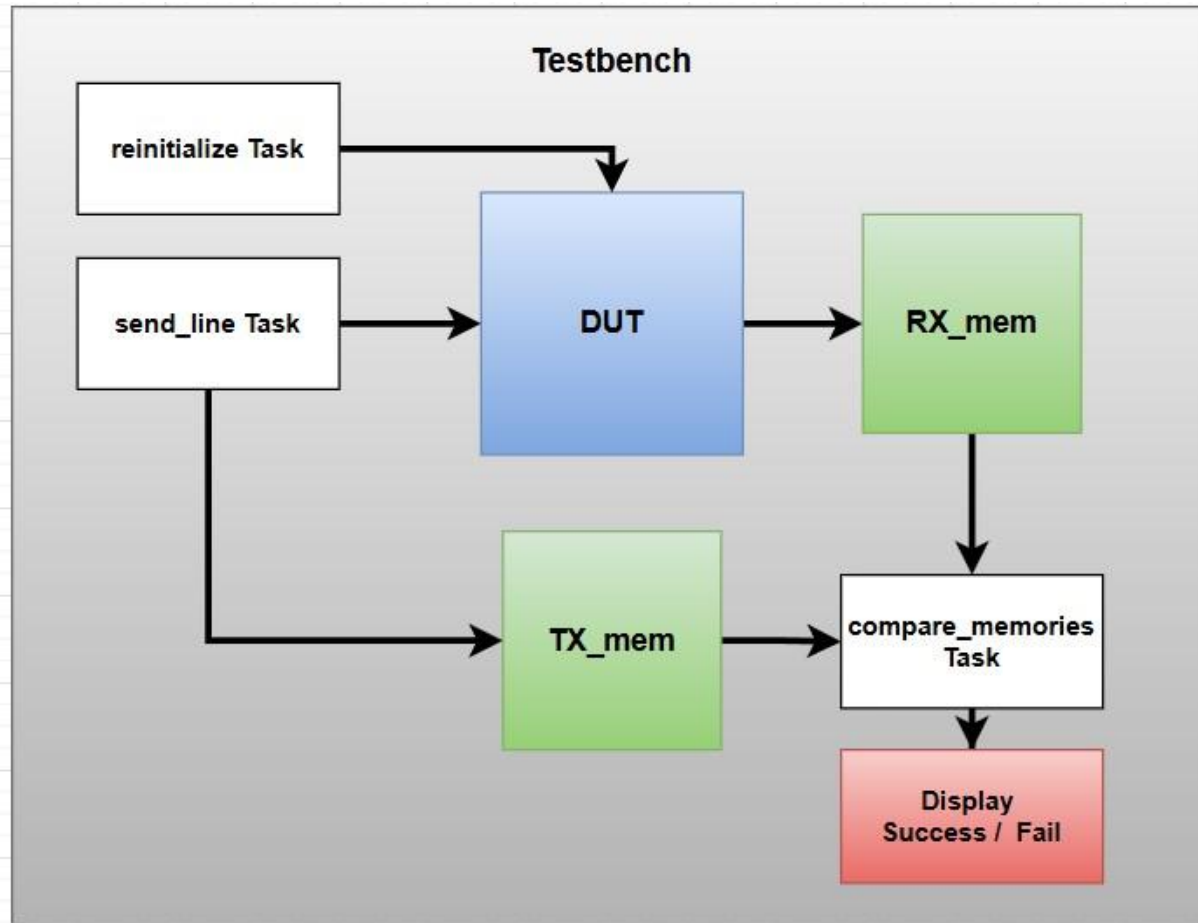


Design and Implementation of CSI Controller IP

System Verilog Testbench



Testbench Architecture & Overview

Objective: End-to-end functional validation of the MIPI CSI-2 Subsystem (TX & RX).

- **Datapath:** AXI-Stream (TX) → CSI-2 TX Controller → Behavioral D-PHY → CSI-2 RX Controller → AXI-Stream (RX).
- **Control Path:** Integrated custom I2C Master for dynamic core configuration and APB Master for feature control.
- **Stimulus Generation:** Highly randomized AXI-Stream pixel generator with automated Start-of-Frame (SOF) and End-of-Line (EOL) packetization.
- **Self-Checking:** Fully automated pass/fail detection without requiring manual waveform inspection

Comprehensive Test Sequence

Objective: Stress-test the DUT across all supported operational modes in a single continuous simulation.

- **Multi-Format Testing:** Automatically sweeps through 5 different video data types: RAW8, RAW10, YUV422, RGB565, and RGB888.
- **Dynamic Lane Switching:** Test sequence dynamically reconfigures the PHY and Controllers to cycle through **4-lane, 2-lane, and 1-lane** operations.
- **Feature Sweeping:** Systematically toggles Payload CRC and Data Scrambling features to guarantee all 4 states are tested (ON/ON, ON/OFF, OFF/ON, OFF/OFF).
- **Volume:** Simulates over 5,000,000 total pixels per run to ensure sustained stability.

Key Verification Tasks

- **send_line Task:** The core AXI-Stream injector. It generates randomized pixel data, dynamically masks bits based on the active Data Type (e.g., 24-bit for RGB888, 8-bit for RAW8), asserts the End-of-Line (tlast) flag, and triggers TX coverage sampling.
- **send_frame Task:** Constructs complete video frames by calling send_line in a loop. It handles intra-line delays and precisely flags the Start-of-Frame (tuser) on the very first pixel.
- **reinit Task:** The dynamic configuration engine. It applies hard and soft resets, calculates payload byte sizes, and reprograms the TX/RX DUT registers via I2C and APB to prepare the hardware for new lane widths, data types, and features.

Key Verification Tasks

- **compare_memories Task:** The automated scoreboard. It cycle-accurately scans the deep tx_mem and rx_mem arrays after every frame to guarantee 100% data match, stopping the simulation instantly if corruption is found.
- **Control Path Tasks:** Custom i2c_write_reg (bit-banging) and apb_write master tasks handle low-level register access.

Scoreboard & Integrity Checking

Objective: Guarantee zero data loss, zero corruption, and accurate protocol tracking.

- **Memory Tracking:** Captures 100% of injected TX AXI-Stream data and extracts RX AXI-Stream data into deep tracking memories.
- **Pixel-by-Pixel Scoreboard:** Performs an automated cycle-accurate comparison between the TX and RX memory arrays.
- **Marker Validation:** Strictly verifies that **tuser** (Start-of-Frame) and **tlast** (End-of-Line) flags arrive on the exact correct pixels.
- **Error Monitoring:** Passively tracks and logs hardware-flagged CRC failures and ECC Single/Double bit errors.

Main Execution Sequence (initial block)

Objective: Fully automate the exhaustive testing of all hardware modes without manual intervention.

- **Initialization Phase:** Declares loop variables, initializes the coverage collector, fills checking memories with background patterns, and applies initial power-on-resets to the system.
- **Data Type Sweep (Outer Loop):** Iterates over an array of 5 video formats (RAW8, RAW10, YUV422, RGB565, RGB888). For each format, it calculates the precise `word_count` needed by the CSI-2 packetizer.
- **Frame Sweep (Inner Loop):** Generates 10 complete video frames per Data Type.

Main Execution Sequence (initial block)

Dynamic Mode Toggling: Inside the frame loop, mathematical operations instantly toggle configurations:

- Modulo arithmetic ($f \% 3$) rotates the PHY between **4-lane, 2-lane, and 1-lane** modes.
- Logic checks ($f \% 2 == 0$) and ($f \% 4 < 2$) generate an alternating Truth Table to exhaustively test the **CRC and Scrambler** combinations (ON/ON, ON/OFF, OFF/ON, OFF/OFF).

Execution Flow: For every frame iteration, it sequentially calls: `reinit()` → `send_frame()` → `compare_memories()`.

Verification Metrics (SVA & Coverage)

Objective: Prove verification completeness and strict protocol adherence.

- **Protocol Monitors (SVA):** Integrated SystemVerilog Assertions seamlessly monitor the bus in the background to catch violations and timing errors.
- **Functional Coverage:** Custom SystemVerilog Covergroups natively embedded into the testbench track the stimulus distribution.
- **Coverage Goals Achieved:** Proven **100% coverage** on Data Types, Lane Widths, Frame Markers, and CRC/Scrambler combinations.

Functional Coverage Model

Objective: Mathematically prove that the testbench generated the necessary stimulus to hit all verification targets.

- **lane_config_cg:** Confirms the testbench successfully simulated the DUT operating in 1-lane, 2-lane, and 4-lane topologies.
- **data_type_cg:** Verifies that AXI-Stream packets were successfully generated for all 5 target data types.
- **frame_ctrl_cg:** Tracks the AXI-Stream frame markers. Ensures Start-of-Frame (sof) and End-of-Line (eol) asserted properly, and uses ignore_bins to ensure they do not illegally collide on the same clock cycle.

Functional Coverage Model

- **features_cg:** Uses a coverage cross between the CRC and Scrambler states. This guarantees the testbench hit all four ON/OFF variations.
- **integrity_cg:** Passively monitors hardware outputs, checking that CRC passes were logged and that no ECC uncorrectable errors were accidentally triggered in an ideal loopback environment.

SystemVerilog Assertions

Objective: Catch low-level timing and protocol violations instantly using continuous background SVA tracking.

- **Pulse Verification (rx_sof_single_beat):** Ensures the Start-of-Frame flag (tuser) does not drag; it must assert for exactly one valid clock cycle at the start of a frame.
- **Reset Compliance (tx_valid_low_after_reset):** Checks that AXI-Stream tvalid signals remain cleanly de-asserted immediately following a system reset.
- **Liveness Properties (rx_stream_terminates):** Prevents hanging buses. Guarantees that an open frame (tvalid) eventually closes with an End-of-Line (tlast) within a strict cycle limit.

SystemVerilog Assertions

- **Integrity Traps (no_ecc_double_error):** An immediate-fail assertion that stops the simulation if the RX hardware ever flags a fatal double-bit error during an ideal, noise-free run.
- **CRC Check:** Verifies that every time the hardware completes a CRC calculation (rx_crc_done goes high), the payload must be error-free (rx_crc_ok is simultaneously high).

(SVA & Coverage)

=== Design Unit: work.csi_coverage_pkg

=====

Covergroup Coverage:

Covergroups	5	na	na	100.00%
Coverpoints/Crosses	11	na	na	na
Covergroup Bins	26	26	0	100.00%

=== Instance: /tb_mipi_system_loopback/u_sva_mon

=== Design Unit: work.csi_sva

=====

Assertion Coverage:

Assertions	5	5	0	100.00%
------------	---	---	---	---------

Synthesis

Design WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

```
79 echo "Starting Incremental Compile for Timing Closure on sys_clk..."
80 group_path -name SYS_CLK_CRITICAL -to [get_clocks sys_clk] -weight 5 -critical_range 0.2
81 compile_ultra -incremental
82
```

```
:8 Total cell area:          685027.619101
:9 Total area:              1414686.257107
:0
```

```
:1 Hierarchical area distribution
```

```
:2 -----
```

```
:3
:4                               Global cell area          Local cell area
:5                               -----
```

```
:6 Hierarchical cell
```

```
:7                               Absolute    Percent    Combi-    Noncombi-    Black-
:8                               Total        Total      national  national    boxes   Design
:9 -----
```

```
:9 mipi_tx_top                685027.6191    100.0    6290.3182    9281.0848    0.0000    mipi_tx_top
```

```
:0 u_axi_tx_u_fifo            669456.2161     97.7    262796.3338  406659.8823  0.0000    async_fifo_behavioral_DEPTH2048_WIDTH26
:1 -----
```

```
:2 Total                      685027.6191    100.0    269086.6520  415940.9671  0.0000
:3
```

```
:4 1
```

```

Cell Internal Power = 59.9042 mW (98%)
Net Switching Power = 1.2224 mW (2%)
-----
Total Dynamic Power = 61.1266 mW (100%)

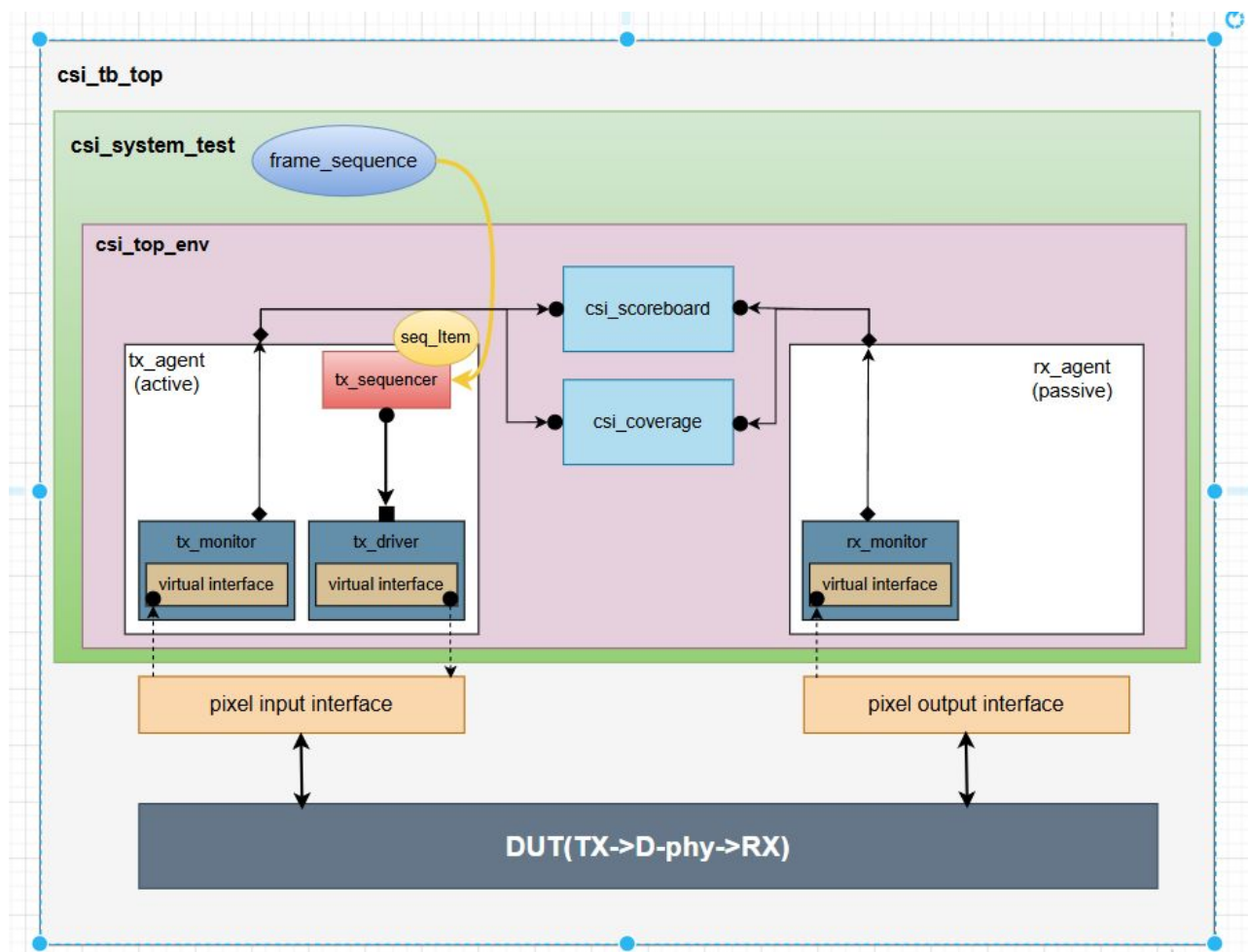
Cell Leakage Power = 63.4865 mW

```

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs

io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	5.8801e+04	0.0000	0.0000	5.8801e+04	(47.18%)	i
register	809.9961	0.3595	4.7539e+10	4.8349e+04	(38.79%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	322.2595	1.2221e+03	1.5948e+10	1.7493e+04	(14.03%)	

UVM



csi_config

Interface Holder: Holds the virtual interface handles (tx_vif, rx_vif) so they can be easily distributed to drivers and monitors.

Agent Control: Defines the operational modes of the agents (e.g., setting the TX agent to UVM_ACTIVE and the RX agent to UVM_PASSIVE).

Global Access: Passed down through the component hierarchy using the uvm_config_db, ensuring all components read from the same configuration.

csi_seq_item (The Data Transaction)

Data Payload: Represents a single AXI-Stream pixel transaction, containing randomized pixel_data dynamically sized for formats like RAW8 or RGB888.

Protocol Markers: Carries the AXI-Stream sideband signals, specifically tuser (Start-of-Frame) and tlast (End-of-Line).

Embedded Configuration: Stores the current state of the DUT (Active Lanes, Data Type, CRC, and Scrambler states) for that specific pixel so the scoreboard and coverage collector know the context of the transaction.

Debug Helpers: Provides custom convert2string() methods to cleanly print transaction details to the simulator transcript.

csi_frame_sequence (The Stimulus Generator)

Automated Packetization: Automatically calculates when to assert tuser (on the very first pixel) and tlast (on the last pixel of every line) based on the configured resolution.

Dynamic Sweeping: Systematically sweeps through combinations of lane counts (4, 2, 1) and Data Types to stress test the PHY bandwidth.

Feature Toggling: Alternates the CRC and Scrambler configurations (ON/ON, ON/OFF, OFF/ON, OFF/OFF) frame by frame.

Pipeline Management: Injects realistic hardware idle times, such as the 3000 pixel_clk inter-line gap and the 5000 byte_clk post-frame drain, ensuring the DUT's packetizer doesn't choke.

csi_rx_agent

Structural Container: Groups the RX-side UVM components together.

Passive Operation: Because it is configured as UVM_PASSIVE, it intentionally omits a sequencer and driver, ensuring it never interferes with the DUT's outputs.

Data Routing: Exposes analysis ports (agt_ap for pixels, stat_ap for hardware events) to broadcast the monitor's findings to the Scoreboard and Coverage collector.

csi_rx_monitor

Pixel Extraction (Thread 1): Passively samples the RX AXI-Stream interface on every valid pixel_clk edge, reconstructing the output video stream.

Hardware Status (Thread 2): Continuously watches the byte_clk domain for CSI-2 hardware integrity pulses (CRC done, CRC ok, ECC single/double bit errors).

Custom Status Items: Generates and broadcasts a dedicated csi_rx_status_item whenever an integrity event occurs, allowing the testbench to separate pixel data from hardware flags.

csi_tx_agent

Structural Wrapper: Groups the TX-side UVM components together into one reusable block.

Active Instantiation: Checks its configuration to confirm it is UVM_ACTIVE, and subsequently builds the csi_tx_sequencer, csi_tx_driver, and csi_tx_monitor.

Port Forwarding: Takes the analysis port from the internal TX Monitor and exposes it at the Agent level (agt_ap) so the environment can connect it to the scoreboard.

csi_tx_monitor

Handshake Sampling: Passively watches the AXI-Stream interface on the TX side, but only captures a pixel when both tvalid and tready are high (confirming the DUT actually accepted the data).

Context Capture: Captures the pixel data, tuser (SOF), and tlast (EOL), while also reading the current DUT configuration (Lane count, Data Type, CRC, Scrambler states) from the virtual interface.

Expected Reference: Broadcasts these perfectly captured input transactions to the scoreboard so it has an exact record of what the DUT is expected to output.

csi_tx_sequencer

Transaction FIFO: A standard UVM sequencer that acts as a holding buffer between the sequence and the driver.

Arbitration: Receives the randomized csi_seq_item objects generated by the csi_frame_sequence and feeds them to the csi_tx_driver one by one whenever the driver asks for the next item.

csi_tx_driver

Multi-Protocol Execution: Translates UVM sequence items into physical pin toggles across three different protocols: I2C (Configuration), APB (Features/Reset), and AXI-Stream (Pixels).

Dynamic Reconfiguration: Executes the reinit() task between frames, calculating exact Word Counts (Bytes) based on the Data Type and pushing them via bit-banged I2C.

State Machine Management: Issues necessary Hard Resets and APB Soft Resets to safely purge the Scrambler LFSRs between frames.

Pipeline Protection: Injects realistic hardware delays (e.g., 3000 pixel_clk inter-line gaps and 5000 byte_clk frame flushes) to prevent the DUT's packetizer from choking on back-to-back data.

csi_scoreboard

FIFO Synchronization: Receives the injected (Expected) stream from the TX monitor and queues it up to account for the DUT's internal processing latency.

Cycle-Accurate Checking: Pops expected pixels from the queue as actual pixels arrive from the RX monitor, to perform comparison on the payload, SOF, and EOL flags.

Integrity Traps: Monitors the status channel and will instantly trigger a UVM_FATAL if the RX hardware flags an uncorrectable ECC double-bit error.

Summary Reporting: accumulates all pixel matches, CRC passes, and ECC events, printing a clean pass/fail summary at the end of the simulation.

csi_coverage

Proof of Execution: Mathematically proves that the stimulus generator actually hit all required test scenarios (Data Types, Lane Configurations, etc.).

Cross Coverage: Uses cross covergroups to ensure every possible combination of CRC and Scrambler states was simulated.

Impossible State Filtering: Uses ignore_bins to filter out physically impossible AXI-Stream states (like SOF and EOL firing on the exact same clock cycle for a 1000-pixel line).

Happy Path Validation: Confirms the testbench executed cleanly by explicitly checking for the *absence* of CRC failures and ECC errors during the loopback run.

csi_sva

A passive hardware monitor containing rules that trigger immediate simulator errors if protocol is violated (e.g., if a frame starts without finishing).

csi_top_env

Component Grouping: The highest-level UVM container that brings together all the major verification blocks (TX Agent, RX Agent, Scoreboard, and Coverage).

TLM Routing: Handles all Transaction Level Modeling (TLM) connections during the connect_phase.

Data Flow: Specifically wires the TX Agent's output to the Scoreboard's "Expected" port, and the RX Agent's output to the Scoreboard's "Actual" and "Status" ports, while also hooking both up to the Coverage collector.

csi_system_test

Entry Point: Acts as the top-level UVM test class that kicks off the simulation.

Configuration Setup: Instantiates the csi_config object, retrieves the virtual interfaces from the uvm_config_db, and propagates the configuration down to the environment.

Sequence Control: Defines the core test scenario in the run_phase by iterating through an array of all 5 supported Data Types (RAW8, RAW10, YUV422, RGB565, RGB888).

Stimulus Injection: Passes resolution parameters (pixels per line, lines per frame) to the csi_frame_sequence and starts it on the TX sequencer.

csi_tb_top

Role: The static SystemVerilog top-level module that connects the physical hardware (DUT) to the dynamic UVM software environment.

- **Clock Generation:** Generates the continuous physical clocks required by the hardware, including the pixel_clk, the high-speed byte_clk, and the sys_clk.
- **Component Instantiation:** Physically wires together the Device Under Test (mipi_system_top), the TX/RX interfaces, and the protocol assertion monitor (csi_sva).
- **Hardware Bootstrapping:** Handles the initial "power-on" reset sequence at time zero to ensure the DUT boots into a clean, known state before the UVM test starts.
- **UVM Launch:** Pushes the physical interface wires into the uvm_config_db (so UVM classes can access them) and executes run_test() to kick off the software simulation.

Coverage and Assertions

Covergroup Coverage:

Covergroups	5	na	na	100.00%
Coverpoints/Crosses	11	na	na	na
Covergroup Bins	23	23	0	100.00%

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/csi_tb_top/u_csi_sva/cover__rx_stream_terminates	csi_sva	Verilog	SVA	csi_sva.sv(109)	41	Covered
/csi_tb_top/u_csi_sva/cover__tx_valid_low_after_reset	csi_sva	Verilog	SVA	csi_sva.sv(97)	1	Covered
/csi_tb_top/u_csi_sva/cover__no_ecc_double_error	csi_sva	Verilog	SVA	csi_sva.sv(84)	16551908	Covered
/csi_tb_top/u_csi_sva/cover__crc_ok_when_done	csi_sva	Verilog	SVA	csi_sva.sv(71)	2003	Covered
/csi_tb_top/u_csi_sva/cover__rx_sof_single_beat	csi_sva	Verilog	SVA	csi_sva.sv(57)	41	Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 5

Total Coverage By Instance (filtered view): 100.00%



Thank You!